PROYECTO INTEGRADOR 1.

Biblioteca escolar.

5 DE NOVIEMBRE DE 2026

YAEL EDAHI ZANABRIGA DIAZ
2126625@uagro.mx

Este proyecto es un sistema sobre una biblioteca escolar, presta libros que no siempre cuentan con la misma cantidad de ejemplares, algunos tienen cinco copias, otros una sola. Se necesita saber cuáles están disponibles, ordenar por cantidad de prestamos y localizarlos por su código ISBN.

La lista de objetos List<Titulo> actuó como la estructura principal para la gestión fluida de algoritmos de ordenamiento y búsqueda, mientras que se aplico el concepto de arreglo escalonado para los ejemplares.

Componentes de la Clase Titulo.

```
9 referencias
public string ISBN {  get; set; }
6 referencias
public string Nombre { get; set; }
5 referencias
public int Prestamos { get; set; }
6 referencias
public int[] Ejemplares { get; set; }
```

ISBN: Código o identificador único del libro.

Nombre: Título del libro.

Prestamos: Contador de veces que se ha prestado un libro.

Ejemplares: Arreglo binario donde 1 representa un ejemplar disponible y 0 un ejemplar prestado.

```
3 referencias
public int EjemplaresDisponibles()
{
    int disponibles = 0;
    foreach (int ejemplar in Ejemplares)
    {
        if (ejemplar == 1)
        {
            disponibles++;
        }
    }
    return disponibles;
}
```

Ejemplares disponibles: Método que recorre el arreglo para contar copias en estado 1.

```
1 referencia
public bool PrestamosEjemplares()
{
    for (int i = 0; i < Ejemplares.Length; i++)
    {
        if (Ejemplares[i] == 1)
        {
            Ejemplares[i] = 0;
            Prestamos++;
            return true;
        }
    }
    return false;
}
}
```

Prestamos ejemplares: Método que recorre el primer ejemplar disponible e incrementa el contador de préstamos.

Ordenamiento por inserción.

```
1 referencia
static void OrdenarInsercion(List<Titulo> lista)
{
    for (int i = 1; i < lista.Count; i++)
    {
        Titulo clave = lista[i];
        int j = i - 1;

        while (j >= 0 && lista[j].Prestamos < clave.Prestamos)
        {
            lista[j + 1] = lista[j];
            j--;
        }
        lista[j + 1] = clave;
    }
}
```

Se ordenaron de mayor a menor según el numero de préstamos. Se tomo el elemento clave a partir del índice 1 y se desplaza a la derecha los elementos que tengan menor numero de préstamos. Insertando clave en su posición adecuada. Su complejidad temporal es de $O(n^2)$ en el peor caso, es ideal para colecciones de tamaño reducido en memoria.

Búsqueda secuencial por Título.

```
2 referencias
static void BuscarPorTitulo(List<Titulo> lista, string tituloBuscado)
{
    bool encontrado = false;

    for (int i = 0; i < lista.Count; i++)
    {
        if (lista[i].Nombre.Equals(tituloBuscado, StringComparison.OrdinalIgnoreCase))
        {
            Console.WriteLine($"Libro encontrado: '{lista[i].Nombre}' (ISBN: {lista[i].ISBN})");
            encontrado = true;
            break;
        }
    }

    if (!encontrado)
    {
        Console.WriteLine($"El libro con titulo '{tituloBuscado}' no fue encontrado en el catalogo.");
    }
}
```

En este bloque de código se localiza un libro por su Nombre ignorando mayúsculas o minúsculas (StringComparison.OrdinalIgnoreCase). Funciona recorriendo la lista de inicio a fin, de i=0 hasta i=n-1, su complejidad temporal es de $O(n)$ en el peor de los casos.

Búsqueda binaria por ISBN.

```
static void BuscarPorISBNBi(List<Titulo> lista, string isbnBuscado)
{
    int inicio = 0;
    int fin = lista.Count - 1;
    bool encontrado = false;
    int comparaciones = 0;

    while (inicio <= fin)
    {
        comparaciones++;
        int medio = inicio + (fin - inicio) / 2;
        int comparacion = string.CompareOrdinal(lista[medio].ISBN, isbnBuscado);

        if (comparacion == 0)
        {
            Console.WriteLine($"Libro encontrado: '{lista[medio].Nombre}' (ISBN: {lista[medio].ISBN} - Comparaciones: {comparaciones})");
            encontrado = true;
            break;
        }

        if (comparacion < 0)
        {
            inicio = medio + 1;
        }
        else
        {
            fin = medio - 1;
        }
    }

    if (!encontrado)
    {
        Console.WriteLine($"El libro con ISBN '{isbnBuscado}' no fue encontrado en el catalogo. - Comparaciones: {comparaciones}");
    }
}
```

Se realizan búsquedas utilizando el estándar ISBN, el catalogo debe de estar ordenado previamente de manera ascendente por ISBN mediante el *catalago.Sort()*. Se divide el intervalo de búsqueda a la mitad evaluando la posición medio = inicio + (fin - inicio) / 2.

Medición y verificación.

Comparaciones.

Se comprobó que, tanto para un elemento existente como para un elemento inexistente, el algoritmo realiza 3 comparaciones, ya que el arreglo es de n=8, la complejidad máxima es $\lceil \log_2(8) \rceil = 3$.

Array.BinarySearch.

Se convirtió el catalogo en un arreglo de string, ejecutándose la función nativa de Framework de .NET.

Para un resultado existente, devuelve el índice 0 que coincide con la búsqueda propia, y cuando el resultado es inexistente, devuelve un valor negativo -1, esto confirma que el elemento no existe en la colección.

Conclusión sobre el proyecto.

El sistema de biblioteca escolar cumple con los requerimientos del día a día para realizarlo. La integración entre las estructuras orientadas a objetos, los métodos de préstamo, el ordenamiento por inserción y la búsqueda demuestra la gestión solida y control fluido de ejecución en C#.